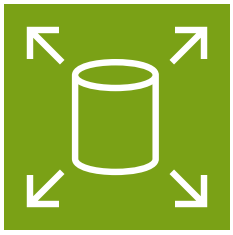
EBS Overview and Volume Types

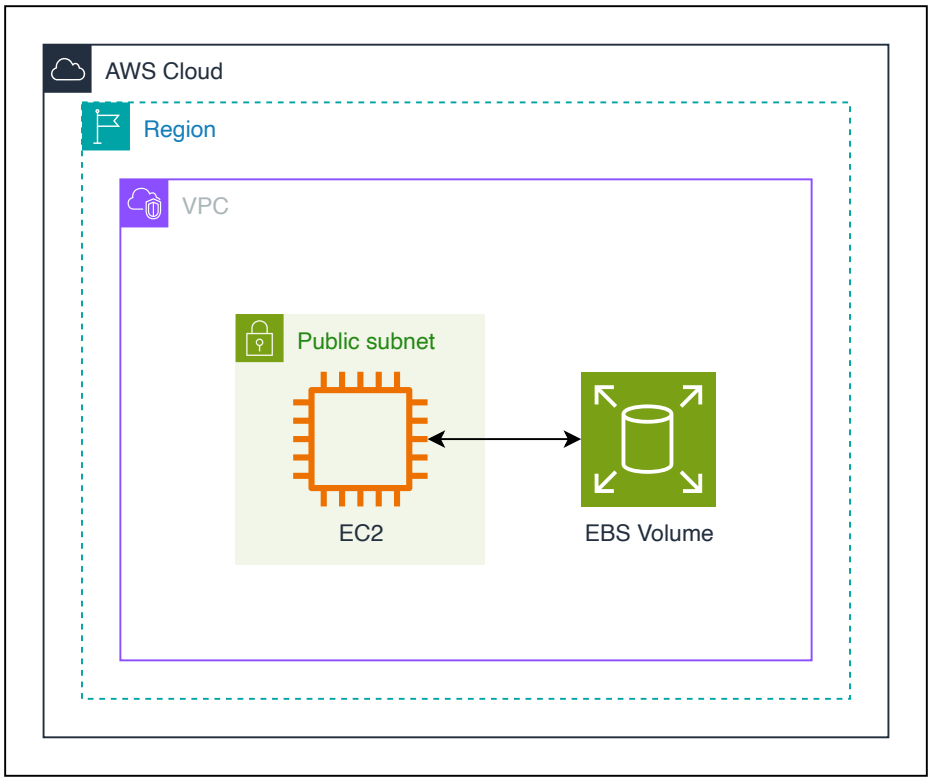
Explore Amazon Elastic Block Store (EBS) and its volume types to understand how to select, manage, and secure block storage for cloud applications. This lesson teaches you to use AWS CLI for provisioning and resizing EBS volumes, monitor performance with CloudWatch, and apply best practices for reliability and cost optimization.

Modern cloud applications need fast, reliable, and scalable storage. Whether it's a database requiring consistent IOPS (input/output operations per second) or a big data application that needs massive throughput, EBS offers the flexibility to tailor storage to our needs. **Amazon Elastic Block Store (EBS)** provides persistent block storage volumes for use with Amazon EC2 instances. Think of EBS volumes as hard drives in the cloud that can scale dynamically with storage requirements. EBS is particularly useful when applications require persistent, low-latency block storage with high durability and availability within a single AZ. For Multi-AZ redundancy or long-term object storage, Amazon S3 may be more appropriate.



EBS volumes are designed to deliver predictable, high-performance storage that can back up everything from simple file systems to complex, transactional databases. One standout feature of EBS is its persistence; even if an EC2 instance stops, the data stored on an EBS volume remains intact.

Ask



EC2 instance connected to an EBS volume

EBS supports features like snapshotting for backups, encryption for security, and Multi-Attach for sharing a single volume across multiple instances (in some cases). This versatility makes it a core building block for AWS-based architectures.

Understanding EBS volume types

AWS provides multiple EBS volume types, each optimized for specific use cases. Understanding their characteristics helps developers select the right type for their application needs.

Volume Type	Baseline Performance	Use Cases	Key Features	Best for
General Purpose SSD (gp2, gp3)	3,000 IOPS and 125 MiB/s (scalable to 16,000	Boot volumes, small to medium databases, and dev/test environments	Predictable performance, independent provisioning of IOPS	Web/app servers, balanced performance-cost

Ask

	IOPS and 1,000 MiB/s)		and throughput, `gp3` has 20% lower cost than `gp2`	needs, default for new volumes
Provisioned IOPS SSD (`io1`)	Up to 64,000 IOPS and 1,000 MiB/s	Large relational/NoSQL databases (e.g., Oracle, SQL Server, and MongoDB)	Custom IOPS provisioning, fine-grained performance control	Mission-critical apps, SAP workloads, high-performance databases
Throughput Optimized HDD (`st1`)	Up to 500 MiB/s throughput	Big data, data warehouses, and log processing	Low-cost, optimized for large, sequential I/O	Workloads focused on sustained throughput rather than low latency
Cold HDD (`sc1`)	Up to 250 MiB/s throughput	Cold data storage, backups, and archives	Most cost-effective, designed for infrequent access	Backup storage, archived files, and disaster recovery with minimal access needs

Comparing volume types

Choosing the right EBS volume is often about balancing performance and cost. Here's a quick side-by-side comparison:

Volume Type	Key Attribute	Performance Focus	Cost
`gp3`	Balanced	IOPS and throughput	Moderate
`io1`	High performance	Maximum IOPS	High
`st1`	Throughput optimized	Large, sequential reads/writes	Low

Ask

`sc1`	Cost optimized	Cold data, minimal throughput	Very low

Managing EBS volumes using AWS CLI

Managing EBS volumes on AWS follows a simple lifecycle: provision, attach, mount, use, unmount, detach, and delete. This structure mirrors real-world workflows and ensures volumes are handled securely and efficiently. By following these steps, we can prevent data loss, support proper resource management, and stay aligned with AWS best practices.

EBS lifecycle

Let's walk through the CLI commands in a real-world lifecycle, where we first attach a volume, mount it to our instance, store data, safely unmount it, and finally detach it when it's no longer needed.

Cre...

```
# Creates a new EBS volume with specified AZ, size, and type.
aws ec2 create-volume \
  --availability-zone <AZ> \
  --size <size> \
  --volume-type <type>
```

Ask

Create an EBS volume

Resizing volumes without downtime

Amazon EBS Elastic Volumes allow us to dynamically modify the size, performance, and volume type of our existing EBS volumes without detaching them. This flexibility supports operational agility by minimizing downtime during changes to our storage requirements.

For instance, if our application suddenly requires more storage or higher IOPS, we can scale up an EBS volume instantly. We can modify volume parameters through the AWS Management Console, AWS CLI, or AWS SDKs. The modifications typically take effect within a few minutes, and for some instance types, even the OS recognizes the expanded size automatically without a reboot.

Here's an example AWS CLI command to modify an existing EBS volume:

main.sh Shell

```
aws ec2 modify-volume \
    --volume-id <Valid_Volume_ID> \
    --size 200 \
    --volume-type gp3 \
    --iops 5000 \
    --throughput 250
```

Ask

Resize the volume

Here's a brief explanation of the resizing command:

- `volume-id`: The ID of the volume we want to modify
- `size`: New size in GiB
- `volume-type`: New volume type if needed
- `iops` and `throughput`: Adjusted performance values for `gp3` volumes

This feature is essential for modern, dynamic cloud applications where storage needs can change unpredictably.

Securing EBS volumes

Security is a crucial aspect of managing EBS volumes. AWS provides encryption options to protect our data at rest. By default, new EBS volumes are encrypted using AWS managed keys through AWS Key Management Service (KMS). However, we can also specify our own customer managed keys (CMKs) for greater control over encryption policies and key rotation.

When creating an EBS volume with a custom CMK, we associate the volume with a key we manage in AWS KMS. Here's an AWS CLI example to create an encrypted EBS volume with a customer managed KMS key:

main.py Python

```
aws ec2 create-volume \
    --availability-zone us-west-2a \
    --size 100 \
    --volume-type gp3 \
    --encrypted \
    --kms-key-id <Valid_Key_ID>
```

Ask

Encrypt EBS volume

Here's a brief explanation of each argument in the AWS CLI command:

- `availability-zone`: It specifies the AZ where the volume will reside.
- `size`: It defines the size of the volume.
- `volume-type`: It chooses the type of EBS volume.
- `encrypted`: It enables encryption.
- `kms-key-id`: It points to the customer managed KMS key used for encryption.

Encrypting our EBS volumes ensures that all data stored at rest and in transit between the instance and the volume is secure. We can attach and mount it just like an unencrypted volume. The encryption process is transparent to our applications.

Troubleshooting EBS volumes

Even with optimal configuration, we may occasionally encounter performance issues with EBS volumes. Effective troubleshooting depends on proactive monitoring and analysis.

To monitor an EBS volume, we can use Amazon CloudWatch metrics to track key indicators like:

- `VolumeReadBytes`, `VolumeWriteBytes` (throughput)
- `VolumeReadOps`, `VolumeWriteOps` (IOPS)
- `VolumeQueueLength` (pending operations)

Ask

We can set up CloudWatch alarms to generate alerts if performance thresholds are breached.

Monitoring EBS alarms

Common performance issues

Here are some common performance issues we encounter:

- **High volume queue length:** This indicates that the volume is receiving more read/write operations than it can handle.
- **Low throughput:** This happens due to exceeding baseline performar limits for the selected volume type.

Ask

- **Burst credits depleted:** Every burstable EBS volume has a baseline performance level that it can consistently deliver. Burst credits deplete when an EBS volume's I/O demand consistently exceeds its baseline performance, consuming credits faster than they can accumulate.

To identify issues, AWS also provides the EBS `VolumeStatusCheck` API, which offers health status, potential degradation, or impaired volume notifications.

By monitoring proactively and selecting the right type and size of volume based on evolving workloads, we can ensure high EBS performance and quickly remediate any degradation.

Best practices

Here are some best practices that should be followed while working with EBS volumes:

- Always enable EBS snapshots for backup and disaster recovery.
- Use encryption for sensitive workloads using AWS KMS managed keys.
- Monitor performance with CloudWatch metrics and X-Ray to avoid bottlenecks.

Conclusion

Amazon EBS provides flexible and powerful storage options for almost any cloud application. By understanding the distinct volume types: `gp3`, `io1`, `st1`, and `sc1`, we can tailor our storage to match our application's performance requirements and budget.

Ask